《计算机图形学及人机交互》

# 实验报告

学　号：2023112621

姓　名：程泽凯

年　级：2023 级

专　业：计算机科学与技术

院　系：计算机与人工智能学院

2025 2026 第 1 学期

# 目录

**1 实验目的、任务及要求** **3**
1.1 实验目的 . . . 3
1.2 实验任务与要求 . . . 3

**2 实验环境** **4**
2.1 硬件环境 . . . 4
2.2 软件环境 . . . 4

**3 主要功能模块结构及说明** **5**
3.1 系统总体架构 . . . 5
3.2 核心模块详细说明 . . . 5
3.2.1 多边形绘制模块 . . . 5
3.2.2 图形变换模块 . . . 6
3.2.3 图形裁剪模块 . . . 7

**4 实验算法及程序流程** **8**
4.1 图形变换算法 . . . 8
4.1.1 平移变换 . . . 8
4.1.2 旋转变换 . . . 9
4.1.3 缩放变换 . . . 10
4.2 Liang-Barsky 直线裁剪算法 . . . 12
4.3 Sutherland-Hodgman 多边形裁剪算法 . . . 15
4.4 Weiler-Atherton 多边形裁剪算法 . . . 17

**5 程序实现** **21**
5.1 算法设计思路 . . . 21
5.1.1 图形变换的设计思路 . . . 21
5.1.2 裁剪算法的设计思路 . . . 21
5.2 核心代码实现 . . . 22
5.2.1 平移变换实现 . . . 22
5.2.2 旋转变换实现 . . . 23
5.2.3 缩放变换实现 . . . 24
5.2.4 Liang-Barsky 直线裁剪实现 . . . 25
5.2.5 Sutherland-Hodgman 多边形裁剪实现 . . . 26
5.2.6 辅助函数实现 . . . 29

5.2.7 多边形工具函数 . . . . . 30
5.2.8 Weiler-Atherton 多边形裁剪实现 . . . . . 32

**6 实验总结** **37**

# 1 实验目的、任务及要求

## 1.1 实验目的

通过本次实验，学习并实现计算机图形学中的基本图形变换和裁剪算法，掌握二维图形的几何变换原理和裁剪技术。

1. 掌握二维图形的基本几何变换：平移、旋转、缩放
2. 理解齐次坐标和变换矩阵的概念
3. 掌握 Liang-Barsky 直线裁剪算法的原理与实现
4. 掌握 Sutherland-Hodgman 多边形裁剪算法的原理与实现
5. 了解 Weiler-Atherton 多边形裁剪算法的基本思想
6. 学习任意多边形的绘制方法

## 1.2 实验任务与要求

1. 任意多边形绘制：实现交互式绘制任意顶点数的多边形
2. 图形平移：实现图形的平移变换，支持鼠标拖动操作
3. 图形旋转：实现图形绑定任意点的旋转变换，支持自定义旋转中心
4. 图形缩放：实现图形绑定任意点的缩放变换，支持自定义缩放中心
5. 直线裁剪：采用 Liang-Barsky 算法实现直线段的裁剪
6. 多边形裁剪：采用 Sutherland-Hodgman 算法和 Weiler-Atherton 算法实现多边形裁剪

# 2 实验环境

## 2.1 硬件环境

1. 计算机：机械革命旷世 16pro
2. CPU：12th Gen Intel® i9-12900H
3. RAM：32GB
4. 显卡：NVIDIA GeForce RTX 4070 Laptop GPU

## 2.2 软件环境

1. 操作系统：Windows 11 专业版
2. 开发工具：Visual Studio 2022
3. 编程语言：C++17
4. 图形 API：Direct2D (Windows SDK)
5. 项目类型：Win32 桌面应用程序

# 3 主要功能模块结构及说明

## 3.1 系统总体架构

本系统基于实验 3 的绘图软件扩展，采用模块化设计。根据实验任务要求，系统主要包括多边形绘制、图形变换（平移、旋转、缩放）和图形裁剪（直线裁剪、多边形裁剪）三大功能模块。

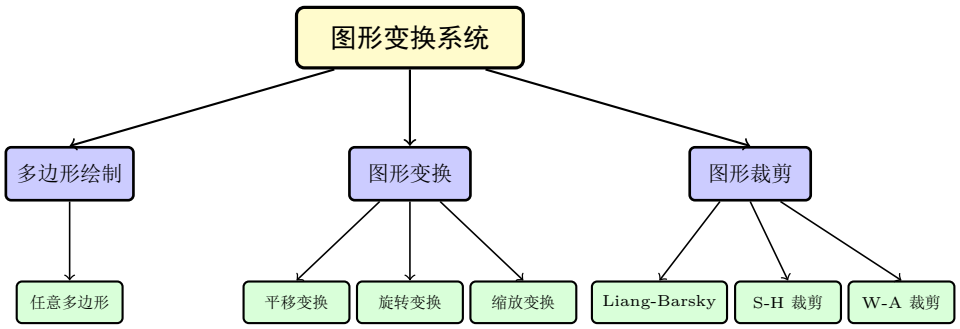



图 1: 系统模块结构图

## 3.2 核心模块详细说明

### 3.2.1 多边形绘制模块

模块功能：
本模块实现任意多边形的交互式绘制功能，用户可以通过鼠标点击依次添加顶点，构建任意形状的多边形。
主要功能：

- 顶点添加：鼠标左键点击添加多边形顶点
- 实时预览：绘制过程中实时显示多边形轮廓
- 自动闭合：右键完成绘制时自动连接首尾顶点
- 填充支持：支持对封闭多边形进行颜色填充

用户交互：

- 选择” 绘图工具 → 自由多边形” 菜单
- 在画布上依次点击添加顶点
- 右键完成绘制，多边形自动闭合

### 3.2.2 图形变换模块

模块功能：
本模块实现二维图形的基本几何变换，包括平移、旋转和缩放三种变换操作。所有变换都基于齐次坐标和变换矩阵实现。
平移变换：

- 将图形沿指定方向移动指定距离
- 变换矩阵：$T(t_x, t_y) = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$
- 用户通过鼠标拖动实现平移操作

旋转变换：

- 将图形绑定任意点进行旋转
- 变换矩阵：$R(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- 支持自定义旋转中心点
- 用户通过鼠标滚轮调整旋转角度

缩放变换：

- 将图形绑定任意点进行缩放
- 变换矩阵：$S(s_x, s_y) = \begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$
- 支持自定义缩放中心点
- 用户通过鼠标滚轮调整缩放比例

用户交互：

- 先选中要变换的图形
- 选择” 变换” 菜单中的平移/旋转/缩放
- 平移：拖动鼠标移动图形
- 旋转/缩放：可先点击设置中心点，然后滚动滚轮调整
- 左键确认变换，右键取消

### 3.2.3 图形裁剪模块

模块功能：
本模块实现图形的裁剪功能，包括直线裁剪和多边形裁剪两类算法。
**Liang-Barsky** 直线裁剪：

- 基于参数化直线方程的裁剪算法
- 通过计算直线与窗口边界的交点参数实现裁剪
- 效率高于 Cohen-Sutherland 算法
- 适用于矩形裁剪窗口

**Sutherland-Hodgman** 多边形裁剪：

- 逐边裁剪算法，依次对四条边界进行裁剪
- 适用于凸多边形裁剪窗口
- 可处理凸多边形和凹多边形
- 输出单个连续的多边形

**Weiler-Atherton** 多边形裁剪：

- 基于交点链表的裁剪算法
- 可处理凹多边形和自相交多边形
- 支持带孔洞的多边形
- 可能输出多个独立的多边形

用户交互：

- 选择” 裁剪” 菜单中的裁剪算法
- 点击选择要裁剪的图形
- 点击选择裁剪矩形
- 系统自动执行裁剪并显示结果

# 4 实验算法及程序流程

## 4.1 图形变换算法

### 4.1.1 平移变换

输入：图形顶点集合 $\{P_i\}$，平移向量 $(t_x, t_y)$
输出：平移后的顶点集合 $\{P_i'\}$
关键变量：$n$= 顶点总数，$i$= 当前索引，$(t_x, t_y)$= 平移向量
详细算法流程：

1. 初始化：$i = 0$，$n =$ 顶点总数
2. 循环（当 $i < n$）：读取 $(x_i, y_i)$，计算 $x_i' = x_i + t_x$，$y_i' = y_i + t_y$，$i = i + 1$
3. 输出：返回 $\{P_i'\}$

变换公式：

$$\begin{pmatrix} x' \\ y' \\ 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$$

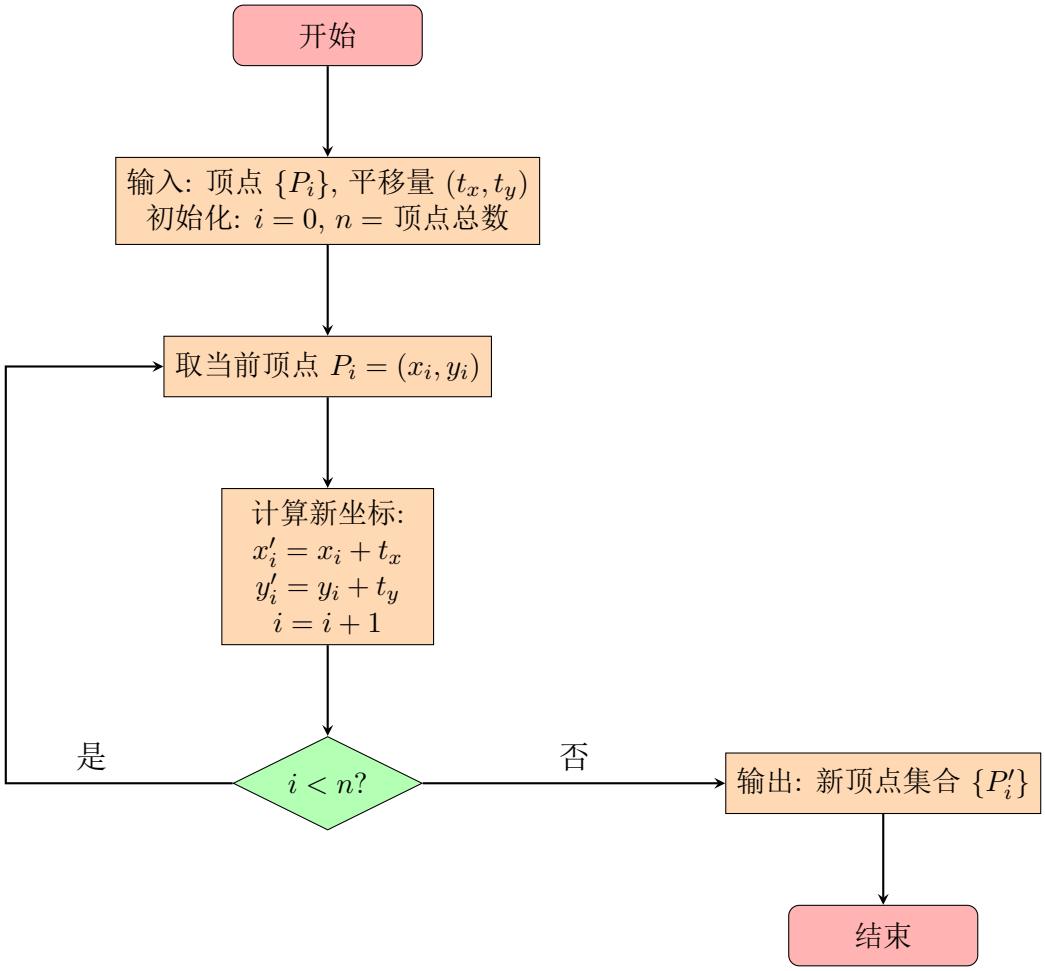



图 2: 平移变换流程图

### 4.1.2 旋转变换

输入：图形顶点集合 $\{P_i\}$，旋转中心 $(c_x, c_y)$，旋转角度 $\theta$
输出：旋转后的顶点集合 $\{P'_i\}$
关键变量：$n$= 顶点数，$(c_x, c_y)$= 旋转中心，$\theta$= 旋转角度，$s = \sin\theta$，$c = \cos\theta$
详细算法流程：

1. 初始化：$i = 0$，$n =$ 顶点数，$s = \sin\theta$，$c = \cos\theta$
2. 循环（当 $i < n$）：
   (a) 平移到原点：$x' = x_i - c_x$，$y' = y_i - c_y$
   (b) 绕原点旋转：$x'' = x' \cdot c - y' \cdot s$，$y'' = x' \cdot s + y' \cdot c$
   (c) 平移回原位：$x'_i = x'' + c_x$，$y'_i = y'' + c_y$
   (d) $i = i + 1$

3. 输出：返回 $\{P_i'\}$

复合变换矩阵：

$$T = T(c_x, c_y) \cdot R(\theta) \cdot T(-c_x, -c_y)$$

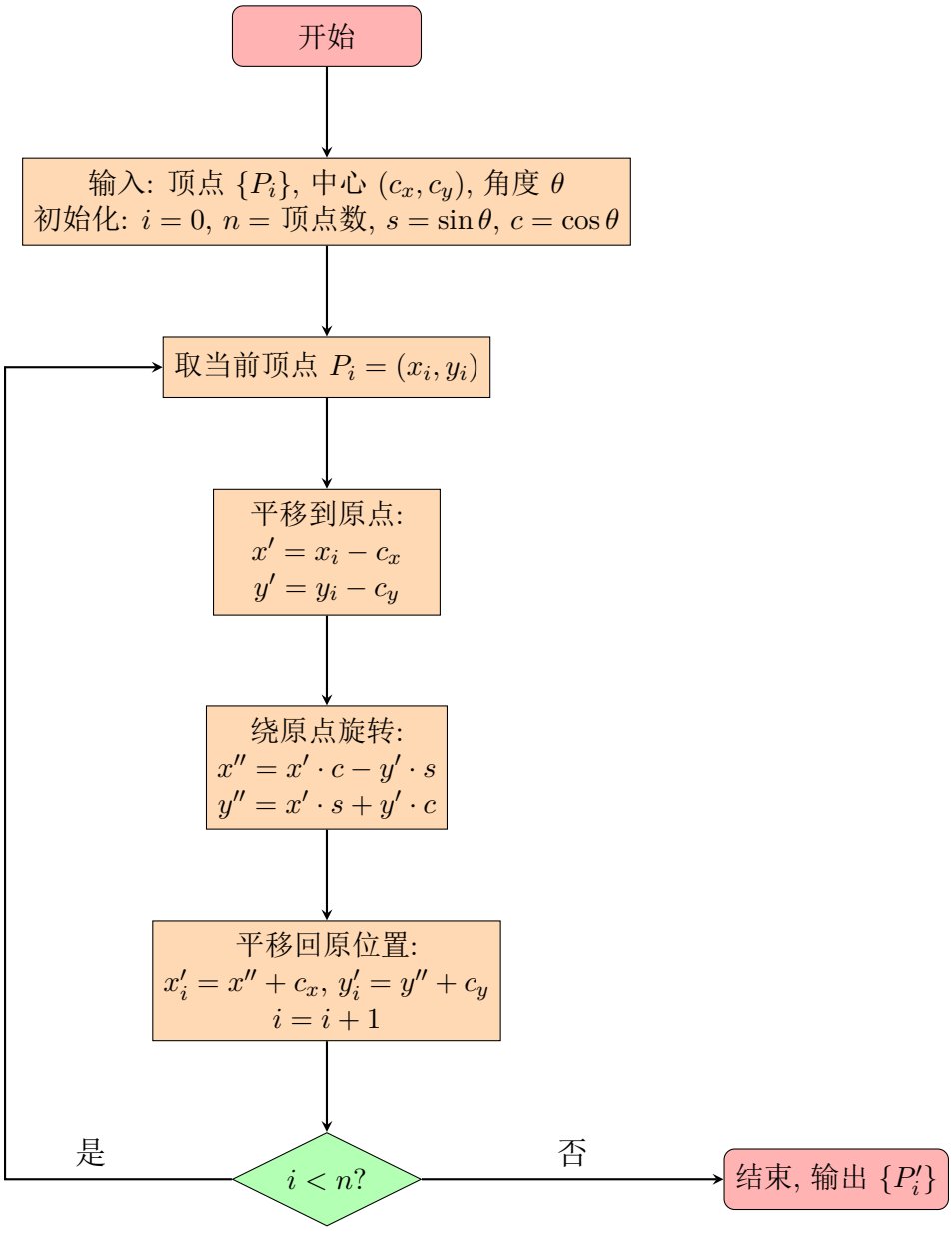



图 3: 旋转变换流程图

### 4.1.3 缩放变换

输入：图形顶点集合 $\{P_i\}$，缩放中心 $(c_x, c_y)$，缩放因子 $s$
输出：缩放后的顶点集合 $\{P_i'\}$

关键变量：$n$= 顶点数，$(c_x, c_y)$= 缩放中心，$s$= 缩放因子
详细算法流程：

1. 初始化：$i = 0$，$n =$ 顶点数
2. 循环（当 $i < n$）：
   (a) 计算相对偏移：$dx = x_i - c_x$，$dy = y_i - c_y$
   (b) 缩放偏移：$dx' = dx \times s$，$dy' = dy \times s$
   (c) 计算新坐标：$x_i' = c_x + dx'$，$y_i' = c_y + dy'$
   (d) $i = i + 1$
3. 输出：返回 $\{P_i'\}$

复合变换矩阵：

$$T = T(c_x, c_y) \cdot S(s, s) \cdot T(-c_x, -c_y)$$

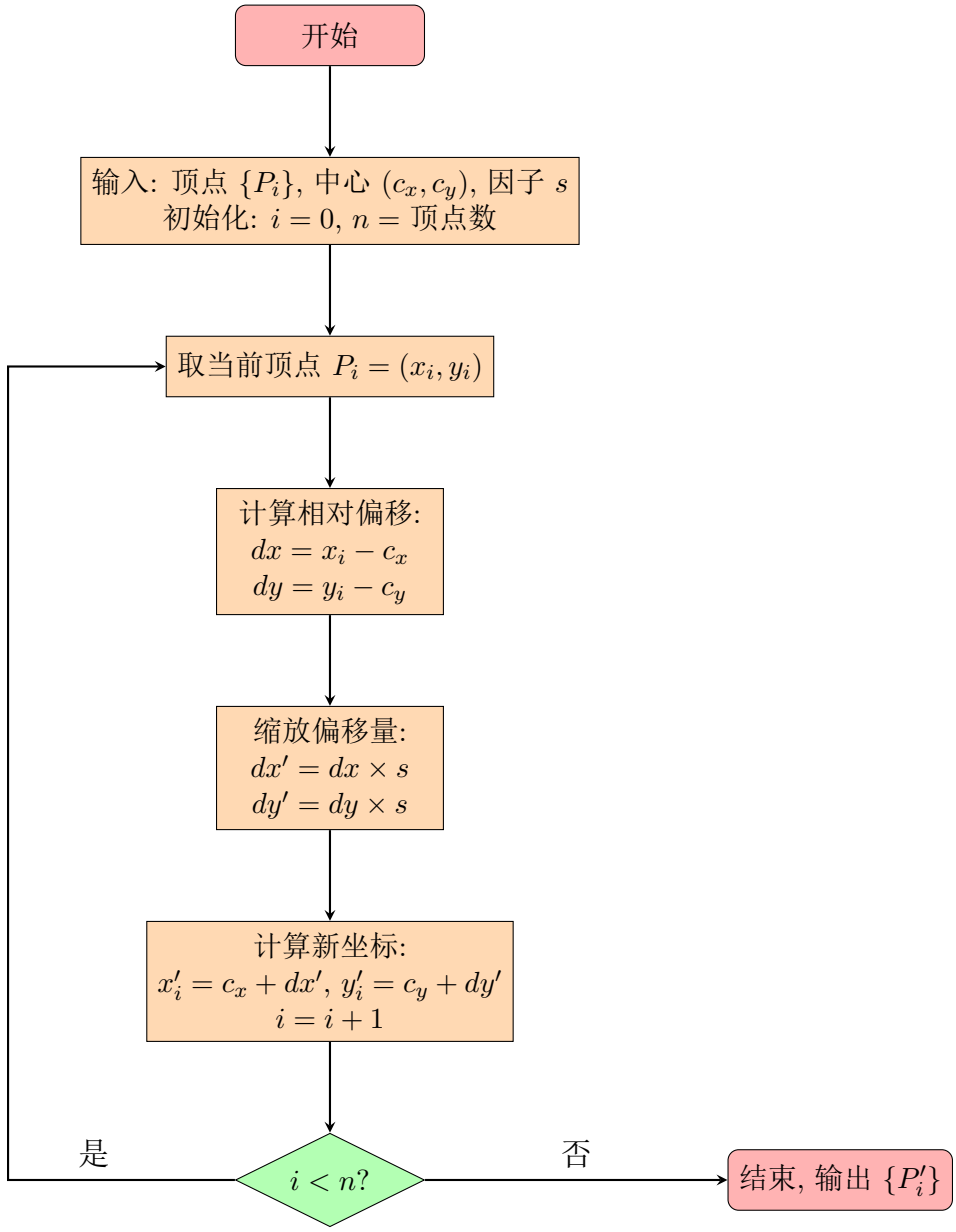



图 4: 缩放变换流程图

## 4.2 Liang-Barsky 直线裁剪算法

输入：直线端点 $(x_1, y_1)$ 和 $(x_2, y_2)$，裁剪窗口 $(x_{min}, y_{min}, x_{max}, y_{max})$
输出：裁剪后的直线端点，或空（直线完全在窗口外）
算法原理：

直线的参数方程为：

$$x = x_1 + t \cdot \Delta x, \quad y = y_1 + t \cdot \Delta y, \quad t \in [0, 1]$$

其中 $\Delta x = x_2 - x_1$，$\Delta y = y_2 - y_1$。
对于四条边界，定义参数 $p_k$ 和 $q_k$：

$$\begin{aligned}
p_1 &= -\Delta x, \quad q_1 = x_1 - x_{min} \quad \text{(左边界)} \\
p_2 &= \Delta x, \quad q_2 = x_{max} - x_1 \quad \text{(右边界)} \\
p_3 &= -\Delta y, \quad q_3 = y_1 - y_{min} \quad \text{(下边界)} \\
p_4 &= \Delta y, \quad q_4 = y_{max} - y_1 \quad \text{(上边界)}
\end{aligned}$$

关键变量：

- $\Delta x = x_2 - x_1$，$\Delta y = y_2 - y_1$：方向向量
- $t_{enter}$：进入参数（初始为 0），$t_{leave}$：离开参数（初始为 1）
- $p_k, q_k$：第 $k$ 条边界的参数（$k = 1, 2, 3, 4$ 对应左/右/下/上）
- $t$：当前边界的交点参数

详细算法流程：

1. 初始化：$\Delta x = x_2 - x_1$，$\Delta y = y_2 - y_1$，$t_{enter} = 0$，$t_{leave} = 1$，$k = 1$
2. 循环（当 $k \leq 4$）：
   (a) 计算 $p_k$ 和 $q_k$（根据边界类型）
   (b) 若 $p_k = 0$ 且 $q_k < 0$：直线平行于边界且在外侧，返回空
   (c) 若 $p_k \neq 0$：计算 $t = q_k / p_k$
   (d) 若 $p_k < 0$（从外到内）：$t_{enter} = \max(t_{enter}, t)$
   (e) 若 $p_k > 0$（从内到外）：$t_{leave} = \min(t_{leave}, t)$
   (f) $k = k + 1$
3. 判断：若 $t_{enter} > t_{leave}$，返回空（无交集）
4. 计算裁剪端点：$(x_1', y_1') = (x_1 + t_{enter} \cdot \Delta x, y_1 + t_{enter} \cdot \Delta y)$，$(x_2', y_2') = (x_1 + t_{leave} \cdot \Delta x, y_1 + t_{leave} \cdot \Delta y)$

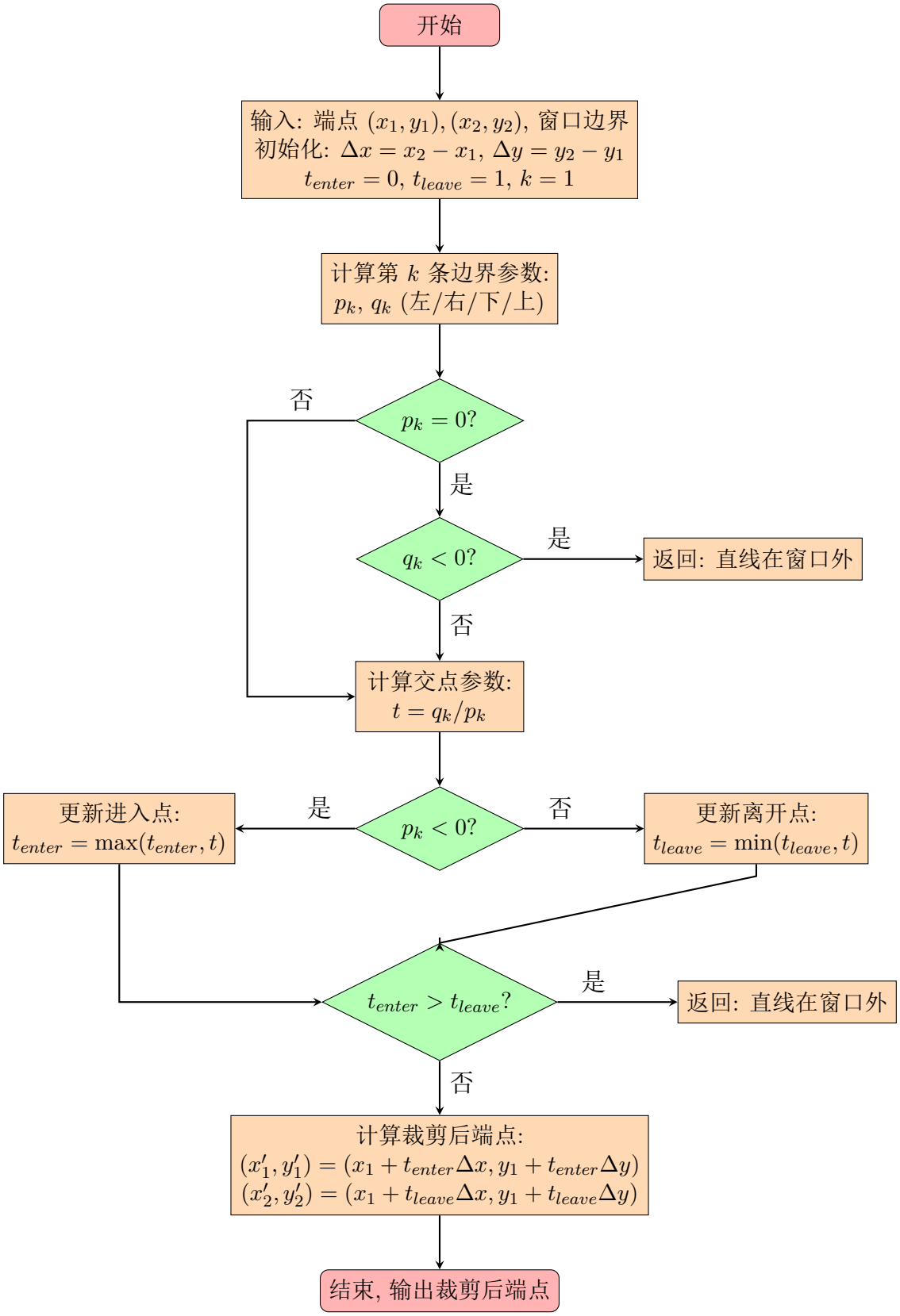



图 5: 优化后的 Liang-Barsky 直线裁剪算法流程图

## 4.3 Sutherland-Hodgman 多边形裁剪算法

输入：多边形顶点列表 $\{P_0, P_1, \ldots, P_{n-1}\}$，裁剪窗口 $(x_{min}, y_{min}, x_{max}, y_{max})$
输出：裁剪后的多边形顶点列表
算法原理：
逐边裁剪算法，依次对裁剪窗口的四条边界进行裁剪。对于每条边界，遍历多边形的每条边，根据边的两个端点与边界的位置关系，决定输出哪些顶点。
关键变量：

- *output*：当前输出多边形顶点列表
- *edge*：当前处理的边界索引（0= 下，1= 右，2= 上，3= 左）
- $P_i, P_{i+1}$：当前处理的边的起点和终点
- $I$：边与边界的交点

四种情况：

1. 两点都在内侧：输出 $P_{i+1}$
2. $P_i$ 内、$P_{i+1}$ 外：计算交点 $I$，输出 $I$
3. 两点都在外侧：不输出
4. $P_i$ 外、$P_{i+1}$ 内：计算交点 $I$，输出 $I$ 和 $P_{i+1}$

详细算法流程：

1. 初始化：$output = \{P_i\}$（输入多边形），$edge = 0$
2. 外循环（当 $edge < 4$）：
   (a) 选择第 *edge* 条边界
   (b) 内循环：对 *output* 中每条边 $(P_i, P_{i+1})$：
      i. 判断 $P_i$ 和 $P_{i+1}$ 与边界的位置关系
      ii. 根据四种情况决定输出顶点
   (c) 更新 *output* 为新的顶点列表
   (d) $edge = edge + 1$
3. 输出：返回 *output*

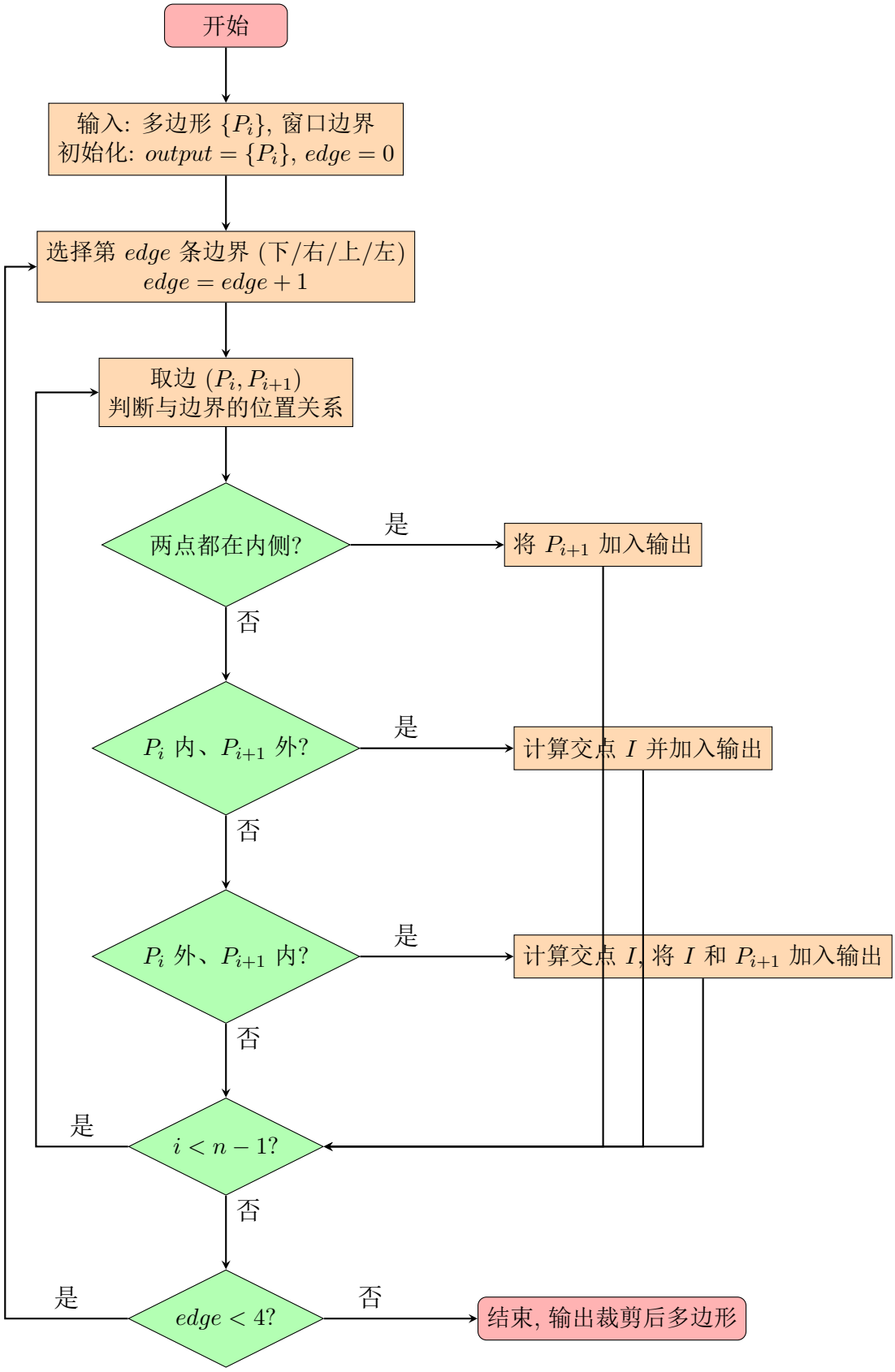



图 6: Sutherland-Hodgman 多边形裁剪算法流程图

## 4.4 Weiler-Atherton 多边形裁剪算法

输入：主多边形（被裁剪多边形）顶点列表 $S = \{S_0, S_1, ..., S_{n-1}\}$，裁剪多边形顶点列表 $C = \{C_0, C_1, ..., C_{m-1}\}$
输出：裁剪后的多边形列表 $R$（凹多边形可能产生多个独立区域）
关键变量说明：

- $subjList$：主多边形顶点链表，包含原始顶点和插入的交点
- $clipList$：裁剪多边形顶点链表，包含原始顶点和插入的交点
- $intersections$：所有交点信息列表，每个交点包含：
  - $pt$：交点坐标 $(x, y)$
  - $t_S$：交点在主多边形边上的参数位置 $t_S \in (0, 1)$
  - $t_C$：交点在裁剪多边形边上的参数位置 $t_C \in (0, 1)$
  - $entering$：是否为进入点（true/false）
- $neighbor$：交点在另一个链表中对应节点的索引（用于切换链表）
- $visited$：标记交点是否已被访问（避免重复遍历）

算法原理：
Weiler-Atherton 算法基于交点链表，可以处理凹多边形、自相交多边形和带孔洞的多边形。
交点分类：

- 进入点 **(Entering)**：主多边形从裁剪多边形外部进入内部的交点
- 离开点 **(Leaving)**：主多边形从裁剪多边形内部离开到外部的交点

详细算法流程：
步骤 **1**：计算所有交点

1. 初始化：$intersections = \emptyset$
2. 对于主多边形的每条边 $e_S = (S_i, S_{(i+1)\%n})$，$i = 0, 1, ..., n-1$：
   (a) 对于裁剪多边形的每条边 $e_C = (C_j, C_{(j+1)\%m})$，$j = 0, 1, ..., m-1$：
   (b) 计算 $e_S$ 与 $e_C$ 的交点，使用参数方程：$P = S_i + t_S \cdot (S_{i+1} - S_i)$
   (c) 若存在有效交点（$0 < t_S < 1$ 且 $0 < t_C < 1$）：

(d) 计算交点坐标：$pt = S_i + t_S \cdot (S_{i+1} - S_i)$

(e) 判断进入/离开：取交点后一小段位置 $pt' = S_i + (t_S + \epsilon) \cdot (S_{i+1} - S_i)$

(f) 若 $pt'$ 在裁剪多边形内部，则为进入点；否则为离开点

(g) 将交点信息加入 $intersections$

步骤 **2**：构建主多边形顶点链表

1. 初始化：$subjList = \emptyset$
2. 对于每个原始顶点 $S_i$，$i = 0, 1, ..., n-1$：
   (a) 将 $S_i$ 加入 $subjList$（标记为非交点）
   (b) 收集边 $(S_i, S_{i+1})$ 上的所有交点，按 $t_S$ 升序排序
   (c) 依次将这些交点插入 $subjList$（标记为交点，设置 $entering$ 属性）

步骤 **3**：构建裁剪多边形顶点链表（同步骤 2，对裁剪多边形执行）

步骤 **4**：建立交点双向链接

1. 对于 $subjList$ 中的每个交点 $I_S$：
   (a) 在 $clipList$ 中找到坐标相同的交点 $I_C$
   (b) 设置 $I_S.neighbor = I_C$ 的索引，$I_C.neighbor = I_S$ 的索引

步骤 **5**：遍历生成裁剪结果

1. 初始化：$R = \emptyset$（结果多边形列表）
2. 对于 $subjList$ 中的每个未访问的进入点 $I_{start}$：
   (a) 初始化当前多边形：$polygon = \emptyset$
   (b) 设置当前位置：$current = I_{start}$，$onSubject = true$
   (c) 循环直到回到起点：
      i. 若 $onSubject = true$（在主多边形上）：
      ii. 将 $current.pt$ 加入 $polygon$
      iii. 标记 $current.visited = true$
      iv. 若 $current$ 是离开点且有链接：
      v. 切换到裁剪多边形：$current = clipList[current.neighbor + 1]$
      vi. 设置 $onSubject = false$

vii. 否则：移动到下一个节点 $current = subjList[(current + 1)\%|subjList|]$

viii. 若 $onSubject = false$（在裁剪多边形上）：

ix. 若 $current$ 是交点且对应主多边形上的进入点：

x. 切换回主多边形：$current = subjList[current.neighbor]$

xi. 设置 $onSubject = true$

xii. 否则：将 $current.pt$ 加入 $polygon$，移动到下一个节点

xiii. 若回到起点 $I_{start}$，退出循环

(d) 若 $|polygon| \geq 3$，将 $polygon$ 加入结果 $R$

3. 返回 $R$

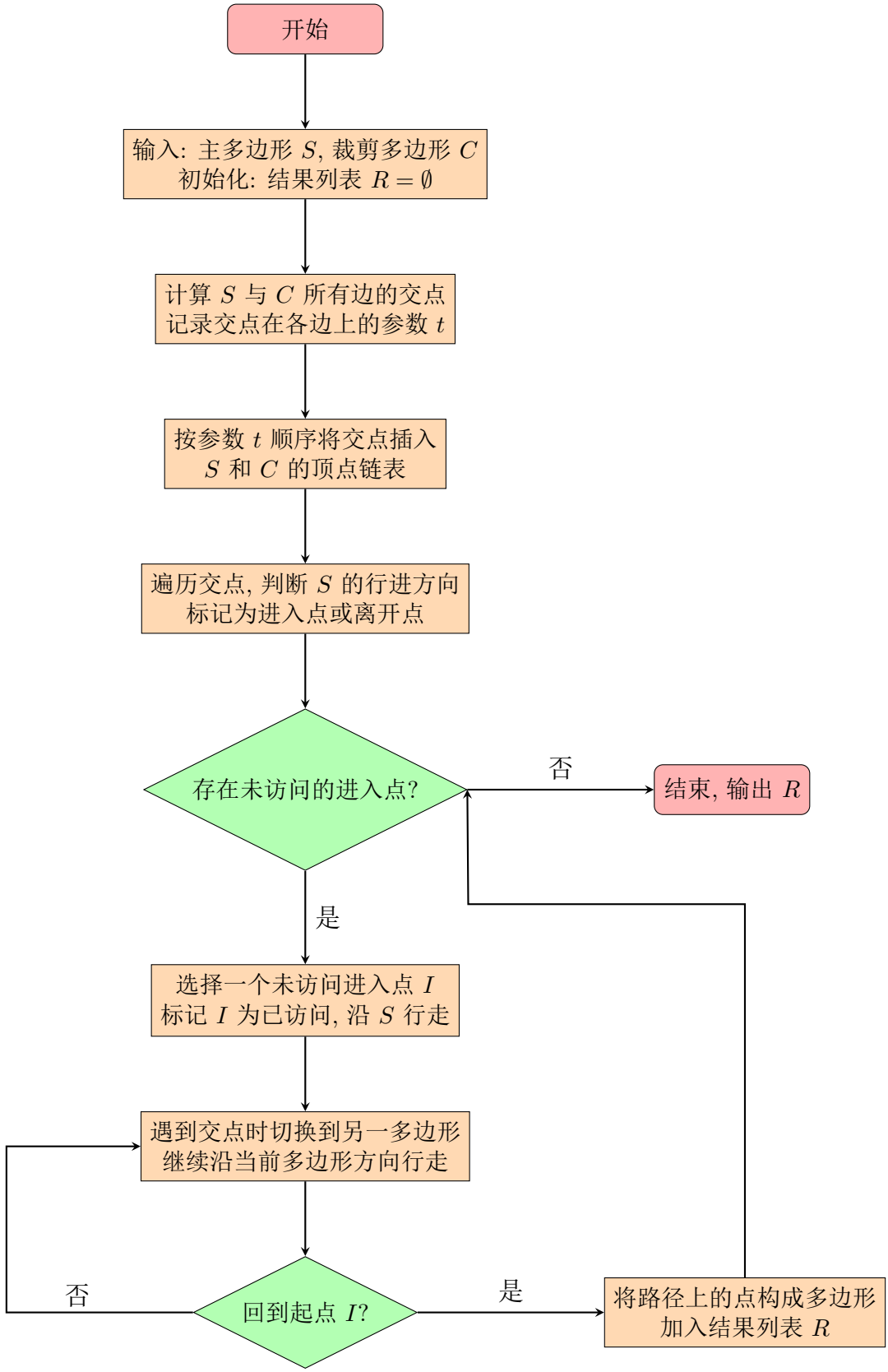



图 7: Weiler-Atherton 多边形裁剪算法流程图

# 5 程序实现

## 5.1 算法设计思路

### 5.1.1 图形变换的设计思路

图形变换的核心思想是将复杂的变换分解为简单的基本操作。对于绑定任意点的旋转和缩放，采用” 三步法”：
设计思路：

1. 问题分析：直接绕任意点变换较为复杂，但绕原点变换的公式简单明了
2. 解决方案：将任意点变换转化为绕原点变换
3. 实现步骤：
    - 第一步：将图形平移，使变换中心移动到原点
    - 第二步：在原点处执行旋转或缩放
    - 第三步：将图形平移回原来的位置

这种设计的优点是代码简洁、易于理解，且可以复用基本变换函数。

### 5.1.2 裁剪算法的设计思路

**Liang-Barsky** 算法思路：

1. 核心思想：将直线表示为参数方程 $P(t) = P_1 + t(P_2 - P_1)$，$t \in [0, 1]$
2. 关键观察：裁剪问题转化为求参数 $t$ 的有效区间
3. 算法策略：
    - 对每条边界计算交点参数 $t$
    - 区分” 进入” 和” 离开” 两种情况
    - 进入点取最大值，离开点取最小值
    - 若进入参数 $>$ 离开参数，则无交集

**Sutherland-Hodgman** 算法思路：

1. 核心思想：分而治之，逐边裁剪
2. 关键观察：对一条边界裁剪后的结果，可以作为下一条边界裁剪的输入

3. 算法策略：依次对四条边界进行裁剪，每次只考虑一条边界的内外关系

**Weiler-Atherton 算法思路：**

1. 核心思想：基于交点链表的遍历
2. 关键观察：裁剪结果的边界由主多边形和裁剪多边形的边交替组成
3. 算法策略：
   - 计算所有交点，标记为” 进入点” 或” 离开点”
   - 从进入点开始沿主多边形行走
   - 遇到离开点时切换到裁剪多边形
   - 遇到进入点时切换回主多边形
   - 直到回到起点，形成一个封闭区域

## 5.2 核心代码实现

### 5.2.1 平移变换实现

代码思路：
平移是最简单的几何变换，只需将每个顶点的坐标加上平移向量即可。
实现要点：

- 平移不改变图形的形状和大小，只改变位置
- 对于多边形，需要遍历所有顶点进行平移
- 平移后需要同步更新用于碰撞检测的 points 数组

```
// ========== 平移变换 ==========
// 输入: dx, dy - 平移向量
// 输出: 无(直接修改图形顶点)
// 算法: 对每个顶点加上平移向量
void Move(float dx, float dy) override {
    // 遍历所有顶点,加上平移量
    for (size_t i = 0; i < polyPoints.size(); i++) {
        polyPoints[i].x += dx;  // x' = x + dx
        polyPoints[i].y += dy;  // y' = y + dy
    }
    points = polyPoints;  // 同步更新points数组
```

```
}
```

Listing 1: 平移变换函数

### 5.2.2 旋转变换实现

代码思路：
绑定任意点的旋转变换采用” 三步法” 实现：先平移使旋转中心到原点，然后绕原点旋转，最后平移回原位置。
实现要点：

- 预先计算 $\sin\theta$ 和 $\cos\theta$，避免在循环中重复计算
- 绕原点旋转使用标准旋转矩阵：$\begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$
- 注意旋转角度使用弧度制，正值为逆时针旋转

```
// ========== 绕任意点旋转变换 ==========
// 输入: p - 待旋转的点坐标
//       center - 旋转中心点坐标
//       angle - 旋转角度(弧度)
// 输出: 旋转后的点坐标
// 算法: 先平移到原点, 旋转后再平移回去
inline D2D1_POINT_2F RotatePoint(D2D1_POINT_2F p,
    D2D1_POINT_2F center, float angle) {
    // 步骤1: 预计算三角函数值(避免重复计算)
    float s = sin(angle);  // sin(theta)
    float c = cos(angle);  // cos(theta)

    // 步骤2: 平移到原点(相对于旋转中心)
    p.x -= center.x;  // x' = x - cx
    p.y -= center.y;  // y' = y - cy

    // 步骤3: 绕原点旋转(应用旋转矩阵)
    // [x'']   [cos -sin] [x']
    // [y''] = [sin  cos] [y']
    float xnew = p.x * c - p.y * s;  // x'' = x'*cos - y'*sin
    float ynew = p.x * s + p.y * c;  // y'' = x'*sin + y'*cos

```

```
    // 步骤4: 平移回原位置
    return D2D1::Point2F(xnew + center.x, ynew + center.y);
}
```

Listing 2: 旋转变换函数

### 5.2.3 缩放变换实现

代码思路：
绑定任意点的缩放变换思路与旋转类似：计算点相对于缩放中心的偏移量，将偏移量乘以缩放因子，再加回中心点坐标。
实现要点：

- 缩放因子 $> 1$ 表示放大，$< 1$ 表示缩小，$= 1$ 表示不变
- 本实现采用等比例缩放（$s_x = s_y$），保持图形比例不变
- 代码将三步合并为一行，提高执行效率

```
// ========== 绕任意点缩放变换 ==========
// 输入: p - 待缩放的点坐标
//       center - 缩放中心点坐标
//       factor - 缩放因子(>1放大, <1缩小)
// 输出: 缩放后的点坐标
// 算法: 计算相对偏移, 缩放后加回中心点
inline D2D1_POINT_2F ScalePoint(D2D1_POINT_2F p,
    D2D1_POINT_2F center, float factor) {
    // 步骤1: 计算相对于中心的偏移量
    // dx = x - cx, dy = y - cy
    // 步骤2: 缩放偏移量
    // dx' = dx * factor, dy' = dy * factor
    // 步骤3: 加回中心点得到新坐标
    // x' = cx + dx', y' = cy + dy'
    return D2D1::Point2F(
        center.x + (p.x - center.x) * factor,  // x' = cx + (x-cx
           )*s
        center.y + (p.y - center.y) * factor   // y' = cy + (y-cy
           )*s
    );
}
```

Listing 3: 缩放变换函数

### 5.2.4 Liang-Barsky 直线裁剪实现

代码思路解析：
Liang-Barsky 算法的核心是利用直线的参数方程。设直线从 $P_1$ 到 $P_2$，则直线上任意一点可表示为：

$$P(t) = P_1 + t \cdot (P_2 - P_1), \quad t \in [0, 1]$$

对于裁剪窗口的四条边界，我们需要找到直线进入窗口和离开窗口的参数值。关键在于：

- 当 $p_k < 0$ 时，直线从边界外侧进入内侧（进入点）
- 当 $p_k > 0$ 时，直线从边界内侧离开到外侧（离开点）
- 当 $p_k = 0$ 时，直线与边界平行

最终，若所有进入点的最大参数值小于所有离开点的最小参数值，则直线与窗口有交集。

```
// ========== Liang-Barsky直线裁剪算法 ==========
// 输入: (x1,y1),(x2,y2) - 直线端点(会被修改)
//       xmin,ymin,xmax,ymax - 裁剪窗口边界
// 输出: true=有交集(端点已更新), false=无交集
// 原理: 直线参数方程 P(t) = P1 + t*(P2-P1), t in [0,1]
static bool ClipLine(float& x1, float& y1, float& x2, float& y2,
    float xmin, float ymin, float xmax, float ymax) {
    // 步骤1: 计算方向向量
    float dx = x2 - x1;  // delta_x = x2 - x1
    float dy = y2 - y1;  // delta_y = y2 - y1

    // 步骤2: 初始化参数区间 [t_enter, t_leave]
    float t_enter = 0.0f;  // 进入窗口的参数(初始为起点)
    float t_leave = 1.0f;  // 离开窗口的参数(初始为终点)

    // 步骤3: 定义四条边界的p和q参数
    // p[k]<0: 从外到内, p[k]>0: 从内到外, p[k]=0: 平行
    float p[4] = { -dx, dx, -dy, dy };       // 左/右/下/上
    float q[4] = { x1-xmin, xmax-x1, y1-ymin, ymax-y1 };
```

```
20
21      // 步骤4: 遍历四条边界,更新参数区间
22      for (int i = 0; i < 4; i++) {
23          if (p[i] == 0) {
24              // 直线平行于该边界
25              if (q[i] < 0) return false;  // 在边界外侧,无交集
26          } else {
27              float t = q[i] / p[i];  // 计算交点参数
28              if (p[i] < 0) {
29                  // 从外到内: 更新进入参数(取较大值)
30                  t_enter = max(t_enter, t);
31              } else {
32                  // 从内到外: 更新离开参数(取较小值)
33                  t_leave = min(t_leave, t);
34              }
35          }
36      }
37
38      // 步骤5: 判断是否有有效交集
39      if (t_enter > t_leave) return false;  // 无交集
40
41      // 步骤6: 计算裁剪后的端点坐标
42      float new_x1 = x1 + t_enter * dx;  // 新起点
43      float new_y1 = y1 + t_enter * dy;
44      float new_x2 = x1 + t_leave * dx;  // 新终点
45      float new_y2 = y1 + t_leave * dy;
46
47      // 步骤7: 更新端点并返回
48      x1 = new_x1; y1 = new_y1;
49      x2 = new_x2; y2 = new_y2;
50      return true;
51  }
```

Listing 4: Liang-Barsky 直线裁剪算法

### 5.2.5 Sutherland-Hodgman 多边形裁剪实现

代码思路：

Sutherland-Hodgman 算法采用” 逐边裁剪” 的策略，将复杂的多边形裁剪问题分解

为四次简单的单边界裁剪。
算法核心思想：

1. 依次对下、右、上、左四条边界进行裁剪
2. 每次裁剪的输出作为下一次裁剪的输入
3. 对于每条边界，遍历多边形的每条边，根据端点位置决定输出

四种情况的处理：

| 情况 | 起点 | 终点 | 输出 | 说明 |
|---|---|---|---|---|
| 1 | 内侧 | 内侧 | 输出终点 | 边完全在内部 |
| 2 | 内侧 | 外侧 | 输出交点 | 边从内到外被截断 |
| 3 | 外侧 | 外侧 | 不输出 | 边完全在外部 |
| 4 | 外侧 | 内侧 | 输出交点和终点 | 边从外到内被截断 |

表 1: Sutherland-Hodgman 四种情况

实现要点：

- 使用取模运算 `(i+1)%size()` 实现多边形的循环遍历
- `IsInside` 函数根据边界类型判断点的内外关系
- `ComputeIntersection` 函数计算线段与边界的交点
- 算法保证输出多边形的连续性和封闭性

```
// ========== Sutherland-Hodgman多边形裁剪算法 ==========
// 输入: polygon - 待裁剪多边形顶点列表
//       xmin,ymin,xmax,ymax - 裁剪窗口边界
// 输出: 裁剪后的多边形顶点列表
// 原理: 逐边裁剪,依次对四条边界进行处理
static std::vector<D2D1_POINT_2F> ClipPolygon(
    const std::vector<D2D1_POINT_2F>& polygon,
    float xmin, float ymin, float xmax, float ymax) {

    if (polygon.size() < 3) return std::vector<D2D1_POINT_2F>();

    std::vector<D2D1_POINT_2F> output = polygon;  // 初始化输出

    // 依次对四条边界进行裁剪(下->右->上->左)
```

```
15      output = ClipAgainstEdge(output, xmin, ymin, xmax, ymin, 0);
            // 下边界
16      output = ClipAgainstEdge(output, xmax, ymin, xmax, ymax, 1);
            // 右边界
17      output = ClipAgainstEdge(output, xmax, ymax, xmin, ymax, 2);
            // 上边界
18      output = ClipAgainstEdge(output, xmin, ymax, xmin, ymin, 3);
            // 左边界
19
20      return output;
21  }
22
23  // ========== 对单条边界进行裁剪 ==========
24  // 输入: input - 当前多边形, edge - 边界类型(0下/1右/2上/3左)
25  // 输出: 裁剪后的多边形
26  // 四种情况: 内内->输出终点, 内外->输出交点
27  //           外外->不输出, 外内->输出交点和终点
28  static std::vector<D2D1_POINT_2F> ClipAgainstEdge(
29      const std::vector<D2D1_POINT_2F>& input,
30      float x1, float y1, float x2, float y2, int edge) {
31
32      std::vector<D2D1_POINT_2F> output;
33      if (input.empty()) return output;
34
35      // 遍历多边形的每条边
36      for (size_t i = 0; i < input.size(); i++) {
37          D2D1_POINT_2F current = input[i];              // 当前顶
                点
38          D2D1_POINT_2F next = input[(i + 1) % input.size()];  //
                下一顶点
39
40          // 判断两个端点与边界的位置关系
41          bool currentInside = IsInside(current, x1, y1, x2, y2,
                edge);
42          bool nextInside = IsInside(next, x1, y1, x2, y2, edge);
43
44          // 根据四种情况决定输出
45          if (currentInside && nextInside) {
```

```
            // 情况1: 两点都在内侧 -> 输出终点
            output.push_back(next);
        } else if (currentInside && !nextInside) {
            // 情况2: 起点内侧,终点外侧 -> 输出交点
            D2D1_POINT_2F intersection = ComputeIntersection(
                current, next, x1, y1, x2, y2, edge);
            output.push_back(intersection);
        } else if (!currentInside && nextInside) {
            // 情况4: 起点外侧,终点内侧 -> 输出交点和终点
            D2D1_POINT_2F intersection = ComputeIntersection(
                current, next, x1, y1, x2, y2, edge);
            output.push_back(intersection);
            output.push_back(next);
        }
        // 情况3: 两点都在外侧 -> 不输出任何点
    }
    return output;
}
```

Listing 5: Sutherland-Hodgman 多边形裁剪算法

### 5.2.6 辅助函数实现

代码思路：
点到线段距离函数用于图形的选中检测（HitTest），判断鼠标点击位置是否靠近图形边界。
算法原理：

1. 将点 $P$ 投影到线段 $AB$ 所在直线上，得到投影点 $P'$
2. 投影参数 $t = \frac{\vec{AP} \cdot \vec{AB}}{|\vec{AB}|^2}$
3. 将 $t$ 限制在 $[0, 1]$ 范围内，确保投影点在线段上
4. 计算点 $P$ 到投影点 $P'$ 的欧氏距离

```
// ========== 点到线段距离 ==========
// 用于图形选中检测(HitTest)
// 输入: p - 测试点, a,b - 线段端点
// 输出: 点到线段的最短距离
```

```
inline float PointToLineDistance(D2D1_POINT_2F p,
    D2D1_POINT_2F a, D2D1_POINT_2F b) {
    float dx = b.x - a.x;
    float dy = b.y - a.y;
    float len = sqrt(dx * dx + dy * dy);

    // 线段退化为点的情况
    if (len < 0.001f)
        return sqrt((p.x-a.x)*(p.x-a.x) + (p.y-a.y)*(p.y-a.y));

    // 计算投影参数t,限制在[0,1]范围内
    float t = ((p.x - a.x) * dx + (p.y - a.y) * dy) / (len * len)
       ;
    t = max(0.0f, min(1.0f, t));

    // 计算投影点坐标
    float px = a.x + t * dx;
    float py = a.y + t * dy;

    // 返回点到投影点的距离
    return sqrt((p.x - px) * (p.x - px) + (p.y - py) * (p.y - py)
       );
}
```

Listing 6: 点到线段距离计算

### 5.2.7 多边形工具函数

代码思路：
这组工具函数为多边形裁剪算法提供基础支持。
**1.** 射线法判断点在多边形内部：

- 从测试点向右发射一条水平射线
- 统计射线与多边形边界的交点数
- 交点数为奇数则在内部，偶数则在外部

**2.** 带符号面积计算：

- 使用向量叉积公式：$A = \frac{1}{2}\sum_{i=0}^{n-1}(x_i y_{i+1} - x_{i+1} y_i)$

- 面积为正表示逆时针，为负表示顺时针
- 可用于判断多边形的绑定方向

```
// 判断点是否在多边形内部（射线法）
static bool PointInPolygon(D2D1_POINT_2F point,
    const std::vector<D2D1_POINT_2F>& polygon) {
    int count = 0;
    size_t n = polygon.size();

    for (size_t i = 0; i < n; i++) {
        D2D1_POINT_2F p1 = polygon[i];
        D2D1_POINT_2F p2 = polygon[(i + 1) % n];

        if ((p1.y <= point.y && p2.y > point.y) ||
            (p2.y <= point.y && p1.y > point.y)) {
            float x = p1.x + (point.y - p1.y) /
                (p2.y - p1.y) * (p2.x - p1.x);
            if (point.x < x) count++;
        }
    }
    return (count % 2) == 1;
}

// 计算多边形面积（带符号，可判断方向）
static float SignedArea(const std::vector<D2D1_POINT_2F>& polygon
    ) {
    float area = 0;
    size_t n = polygon.size();
    for (size_t i = 0; i < n; i++) {
        size_t j = (i + 1) % n;
        area += polygon[i].x * polygon[j].y;
        area -= polygon[j].x * polygon[i].y;
    }
    return area / 2.0f;
}

// 判断多边形是否为顺时针方向
static bool IsClockwise(const std::vector<D2D1_POINT_2F>& polygon
    ) {
```

```
35      return SignedArea(polygon) < 0;
36  }
```

Listing 7: 多边形工具函数

### 5.2.8 Weiler-Atherton 多边形裁剪实现

代码思路：
Weiler-Atherton 算法是最复杂但也最强大的多边形裁剪算法，能够处理凹多边形并输出多个独立区域。
算法五个步骤：
步骤 **1 -** 计算所有交点：遍历主多边形和裁剪多边形的所有边对，计算交点坐标和参数位置。
步骤 **2 -** 判断进入/离开：对每个交点，检查交点后一小段位置是否在裁剪区域内部：

- 若在内部，则为” 进入点”（从外到内）
- 若在外部，则为” 离开点”（从内到外）

步骤 **3 -** 构建顶点链表：将交点按参数 $t$ 顺序插入到两个多边形的顶点链表中。
步骤 **4 -** 建立双向链接：为每个交点建立指向另一个链表中对应交点的链接，用于遍历时切换。
步骤 **5 -** 遍历生成结果：

- 从一个未访问的进入点开始
- 沿主多边形方向行走，收集路径上的点
- 遇到离开点时，通过链接切换到裁剪多边形
- 沿裁剪多边形行走，直到遇到进入点
- 通过链接切换回主多边形，继续行走
- 回到起点时，完成一个裁剪区域

这种” 交替行走” 的策略使得算法能够正确处理凹多边形产生的多个独立区域。
实现要点：

- 使用 `Vertex` 结构体存储顶点信息，包括坐标、参数、类型标记等
- `neighbor` 字段实现两个链表之间的双向链接

- visited 字段避免重复遍历同一个交点
- 判断进入/离开时，取交点后 $\epsilon = 0.001$ 处的点进行测试
- 结果可能包含多个独立的多边形区域

```
// ========== Weiler-Atherton多边形裁剪算法 ==========
// 输入: subject - 主多边形(被裁剪), clipPoly - 裁剪多边形
// 输出: 裁剪后的多边形列表(可能有多个)
// 原理: 基于交点链表,从进入点遍历生成裁剪结果

// 顶点节点结构
struct Vertex {
    D2D1_POINT_2F pt;    // 顶点坐标
    float t;             // 交点在边上的参数位置
    bool isIntersection; // 是否为交点
    bool isEntering;     // true=进入点, false=离开点
    bool visited;        // 是否已访问
    int neighbor;        // 对应另一链表中的索引
};

static std::vector<std::vector<D2D1_POINT_2F>> ClipPolygons(
    const std::vector<D2D1_POINT_2F>& subject,
    const std::vector<D2D1_POINT_2F>& clipPoly) {

    std::vector<std::vector<D2D1_POINT_2F>> result;

    // ===== 步骤1: 计算所有交点 =====
    std::vector<IntersectionInfo> intersections;
    for (size_t i = 0; i < subject.size(); i++) {
        D2D1_POINT_2F s1 = subject[i];
        D2D1_POINT_2F s2 = subject[(i + 1) % subject.size()];

        for (size_t j = 0; j < clipPoly.size(); j++) {
            D2D1_POINT_2F c1 = clipPoly[j];
            D2D1_POINT_2F c2 = clipPoly[(j + 1) % clipPoly.size()
                ];

            float ts, tc; D2D1_POINT_2F pt;
```

```
            if (ComputeIntersection(s1, s2, c1, c2, ts, tc, pt))
                {
                // 判断进入/离开: 检查交点后一小段是否在裁剪区域内
                D2D1_POINT_2F afterPt = {
                    s1.x + (ts + 0.001f) * (s2.x - s1.x),
                    s1.y + (ts + 0.001f) * (s2.y - s1.y)
                };
                bool entering = PointInsidePolygon(afterPt, clipPoly);
                intersections.push_back({pt, i, j, ts, tc, entering});
            }
        }
    }

    // ===== 步骤2: 构建主多边形顶点链表(插入交点) =====
    std::vector<Vertex> subjList;
    for (size_t i = 0; i < subject.size(); i++) {
        Vertex v; v.pt = subject[i]; v.isIntersection = false;
        subjList.push_back(v);
        // 收集该边上的交点,按参数t排序后插入
        // ... (按t_S升序插入交点)
    }

    // ===== 步骤3: 构建裁剪多边形顶点链表 =====
    std::vector<Vertex> clipList;
    // ... (同步骤2)

    // ===== 步骤4: 建立交点双向链接 =====
    for (size_t i = 0; i < subjList.size(); i++) {
        if (subjList[i].isIntersection) {
            for (size_t j = 0; j < clipList.size(); j++) {
                if (clipList[j].isIntersection) {
                    // 找到坐标相同的交点,建立链接
                    float dx = subjList[i].pt.x - clipList[j].pt.x;
                    float dy = subjList[i].pt.y - clipList[j].pt.
```

```
                        y;
                    if (dx*dx + dy*dy < 1e-4f) {
                        subjList[i].neighbor = j;
                        clipList[j].neighbor = i;
                        break;
                    }
                }
            }
        }

    // ===== 步骤5: 从进入点遍历生成裁剪结果 =====
    for (size_t start = 0; start < subjList.size(); start++) {
        // 只从未访问的进入点开始
        if (!subjList[start].isIntersection ||
            !subjList[start].isEntering ||
            subjList[start].visited) continue;

        std::vector<D2D1_POINT_2F> polygon;
        int current = start;
        bool onSubject = true;  // 当前在主多边形上

        do {
            if (onSubject) {
                Vertex& v = subjList[current];
                polygon.push_back(v.pt);
                v.visited = true;

                // 若是离开点,切换到裁剪多边形
                if (v.isIntersection && !v.isEntering && v.
                    neighbor >= 0) {
                    current = (v.neighbor + 1) % clipList.size();
                    onSubject = false;
                } else {
                    current = (current + 1) % subjList.size();
                }
            } else {
                Vertex& v = clipList[current];
```

```
                // 若是进入点,切换回主多边形
                if (v.isIntersection && v.neighbor >= 0 &&
                    subjList[v.neighbor].isEntering) {
                    current = v.neighbor;
                    onSubject = true;
                } else {
                    polygon.push_back(v.pt);
                    current = (current + 1) % clipList.size();
                }
            }
        } while (!(onSubject && current == start));

        if (polygon.size() >= 3) result.push_back(polygon);
    }
    return result;
}
```

Listing 8: Weiler-Atherton 多边形裁剪算法核心实现

# 6 实验总结

通过本次实验，深入理解了计算机图形学中图形变换和裁剪算法的原理和实现方法：

1. 齐次坐标与变换矩阵：使用齐次坐标可以将平移、旋转、缩放等变换统一用矩阵乘法表示，便于复合变换的计算。绑定任意点的旋转和缩放需要先平移到原点，变换后再平移回去。

2. **Liang-Barsky** 算法：该算法基于参数化直线方程，通过计算直线与窗口边界的交点参数来确定裁剪结果。相比 Cohen-Sutherland 算法，Liang-Barsky 算法减少了交点计算次数，效率更高。

3. **Sutherland-Hodgman** 算法：该算法采用逐边裁剪的策略，依次对四条边界进行裁剪。算法简单易实现，但只能输出单个连续的多边形，对于产生多个独立区域的情况无法正确处理。

4. **Weiler-Atherton** 算法：该算法基于交点链表，可以处理凹多边形、自相交多边形和带孔洞的多边形，能够输出多个独立的多边形。算法较为复杂，但功能更加强大。

5. 用户交互设计：良好的用户交互设计可以大大提高软件的易用性。本实验实现了鼠标拖动平移、滚轮旋转/缩放、自定义中心点等交互方式，使用户操作更加直观便捷。

本实验加深了对计算机图形学变换和裁剪算法的理解，提高了使用 Direct2D 进行图形编程的能力。